

Lab 4: EDA - Univariate Analysis

Author: Date: 2026-04-20

Topic Outline:

- Exploratory Data Analysis:
 - Univariate analysis:
 - Producing and interpreting summary statistics for variables.
 - Plots and visualizations for understanding variable distribution.
 - Multiplots.
-

Collaboration

You are encouraged to collaborate with other students in your labs, but you are expected to write up your own work for submission. Please do not copy and paste other people's solutions to problems as it is considered plagiarism and you will be penalized and reported. Should you choose to collaborate with others, please note their names here:

Your name: Chloe Lo Papa

Collaborators:

1.

Agent Usage

Additionally, you are permitted to use resources such as ChatGPT and Claude to help you with your lab assignments and to enhance your learning experience. Please make a note of any agents you have used in this submission here:

Agents:

1. Introduction

We now know how to load, prepare and visualize data. In this lab we will practice implementing all of these skills to perform **exploratory data analysis**, the

process of summarizing variable distributions and relationships. Your focus throughout this lab should not be producing the visualizations or computing the statistics, but instead understanding what information they are providing you. Good exploratory analysis should be thorough, efficient, and well communicated.

In the next chunk we load the required data packages:

```
In [1]: # Packages
import numpy as np
import pandas as pd
import missingno as msno
import matplotlib.pyplot as plt
import seaborn as sns

# Seaborn style
sns.set_style("whitegrid")
sns.set_palette("Set2")
```

In this lab we will be exploring a synthetic data set

`student_performance.csv` which contains information on the academic performance of students. This dataset was provided by [Mubashir Ahmed Siddiqui on Kaggle](#). You can download the `.csv` file from **Canvas** and load into Python using the chunk below:

```
In [14]: # Load data
df = pd.read_csv(
    'student_performance.csv',
    keep_default_na=False                                # this option prevents "N
)
# Change gender and parent_education to categorical
df[['gender', 'parent_education']] = df[['gender', 'parent_education']]
```

This data has been prepared in advance with the details provided in the lab solutions. We inspect the first few rows of this prepared data set using `shape` and examine the variable properties using `info`.

```
In [15]: # Look at data information
print(df.info())
# Examine first few rows
df.head()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   student_id                            500 non-null    str
1   gender                                500 non-null    category
2   age                                    500 non-null    int64
3   study_hours_per_week                  500 non-null    int64
4   attendance_rate                        500 non-null    float64
5   parent_education                      500 non-null    category
6   internet_access                       500 non-null    bool
7   extracurricular                       500 non-null    bool
8   previous_score                        500 non-null    int64
9   final_score                           500 non-null    int64
10  passed                                500 non-null    bool
dtypes: bool(3), category(2), float64(1), int64(4), str(1)
memory usage: 26.1 KB
None

```

```

Out[15]:
  student_id  gender  age  study_hours_per_week  attendance_rate  parent_ec
0   STU0001    Male   15                    25             63.8
1   STU0002  Female   15                    2             54.7
2   STU0003  Female   19                    10             90.5    Hig
3   STU0004    Male   16                    26             66.8    Hig
4   STU0005  Female   15                    25             73.0    Hig

```

We have a selection of categorical and numerical variables and we are interested in exploring which variables influence students passing the class. We note that it is likely that the class has some passing final grade in which case the `passed` variable is deterministically given by `final_score` as

$$\text{passed} = 1(\text{final_score} > c),$$

for some unknown c where 1 denotes an indicator function taking a value of 1 when the condition is satisfied and 0 otherwise. It is important that we confirm this first as this would mean that these variables cannot be used to model each other.

2. Exploratory Data Analysis

2.1 Final Score and Passing

As noted in Section 1, it is likely that students pass the class if they obtain a sufficiently high final score. To check we split our data into failing and passing students and compute the maximum and minimum final score respectively.

```
In [16]: min_pass = df[df['passed']]['final_score'].min().item()
max_fail = df[~df['passed']]['final_score'].max().item()
print(f"Minimum final score for passing students: {min_pass}")
print(f"Maximum final score for failing students: {max_fail}")
```

```
Minimum final score for passing students: 50
Maximum final score for failing students: 49
```

We see that students pass the class if they obtain a score ≥ 50 . Although `passed` is completely determined by `final_score` we leave both in the data frame to be used as comparisons for categorical and quantitative variables respectively. We begin by conducting our **univariate analysis** with the aim of understanding the **distributions** of each variable.

2.2 Univariate Analysis

2.2.1 Categorical Variables

We start our univariate analysis by considering **categorical variables**. As discussed in lecture, for categorical variables our exploration typically consists of tabulating category counts, proportions, and percentages. In our first short problem we practice our skills at defining functions to compute these measurements for specified variables of a data frame.

!! Problem

Q1 Categorical Tabulation

Write a function called `categorical_summary(df, col)` where:

1. `df` specifies the data frame we are considering.
2. `col` specifies the column we are selecting.

The function should return a data frame with rows corresponding to the categories of the selected column and with columns `Count`, `Proportion`, and `Percentage`, which should contain the count, proportion, and percentage of observations for each category value.

Test the function on two categorical columns of your choosing.

Solution:

```
In [ ]: def categorical_summary(df, column):
        counts = df[column].value_counts()
        proportions = df[column].value_counts(normalize=True)
        percentages = proportions * 100
        summary_df = pd.DataFrame({
            'Count': counts,
            'Proportion': proportions,
            'Percentage': percentages
        })
        return summary_df

print(categorical_summary(df, 'study_hours_per_week'))
print(categorical_summary(df, 'attendance_rate'))
```

	Count	Proportion	Percentage
attendance_rate			
72.7	5	0.010	1.0
83.5	5	0.010	1.0
85.2	4	0.008	0.8
70.6	4	0.008	0.8
59.0	4	0.008	0.8
...	...	...	...
77.0	1	0.002	0.2
70.2	1	0.002	0.2
57.8	1	0.002	0.2
69.7	1	0.002	0.2
59.1	1	0.002	0.2

[326 rows x 3 columns]

Although informative, producing multiple tables is inefficient and slow to read. Instead, let us produce a multiplot of count plots for each categorical variable. First, to avoid repetitive filtering, we start by isolating categorical variables into the new data frame `df_cat`:

```
In [21]: # Isolate categorical and boolean variables as a new data frame
df_cat = (
    df.select_dtypes(include=["object", "category", "str", "bool"])
    .copy()
    .drop(columns=['student_id'])
)
df_cat.head()
```

Out [21]:

	gender	parent_education	internet_access	extracurricular	passed
0	Male	Bachelor	True	True	True
1	Female	Bachelor	True	True	False
2	Female	High School	True	False	False
3	Male	High School	False	True	True
4	Female	High School	False	True	True

Next, we produce the multiplot using the `FacetGrid` function from `seaborn`. This syntax is more complicated, so please **spend some time studying the syntax and reading the comments outlining what each line is doing**. You will be asked to produce a similar multiplot for box plots when we consider numerical variables.

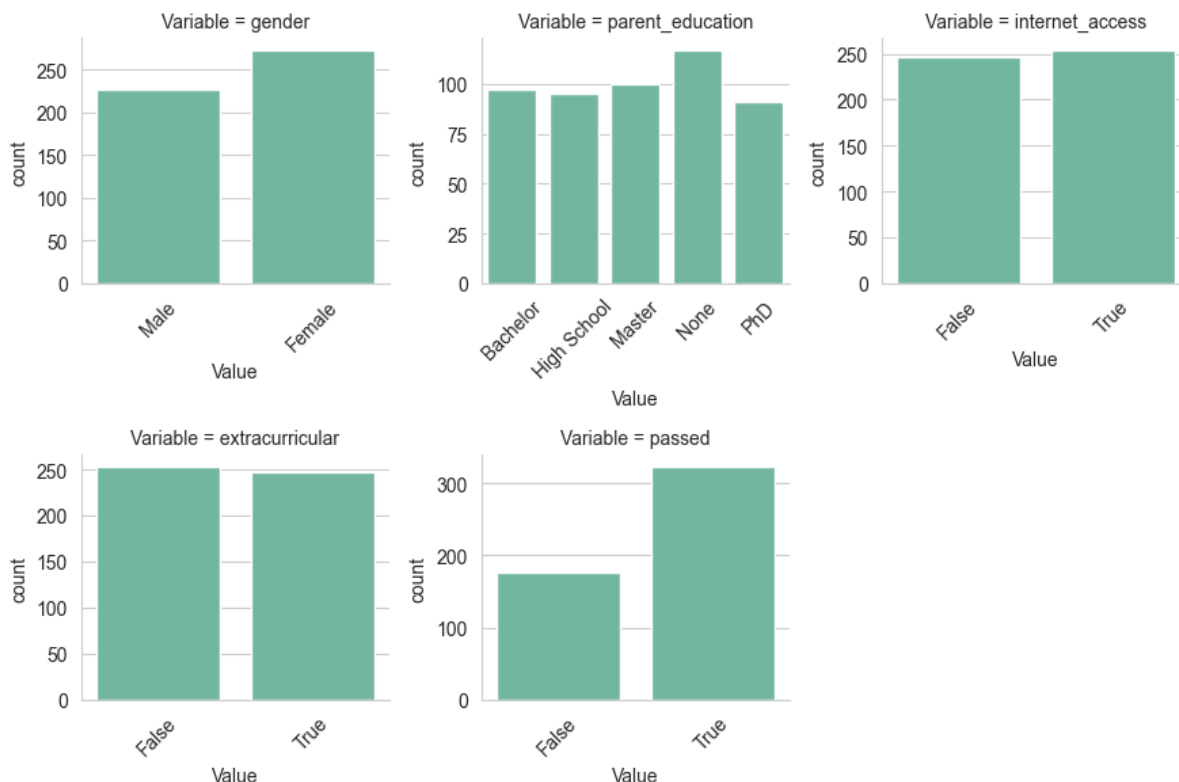
```
In [22]: # Use melt to put data into long form
# Each row becomes one (variable, value) pair for faceting.
df_cat_melt = df_cat.melt(
    var_name='Variable', # Name of the new column that stores original variable names
    value_name='Value'   # Name of the new column that stores categorical values
)

# Helper function that draws one countplot per facet.
def facet_countplot(data, color=None, **kwargs):
    sns.countplot(
        data=data,           # Subset of rows passed in by FacetGrid for faceting.
        x='Value',           # Categorical values go on the x-axis.
        color=color,         # Single color provided by FacetGrid/palette.
        **kwargs             # Accept extra plotting args from map_dataframe
    )

# Build a grid of subplots, one panel per categorical variable.
g = sns.FacetGrid(
    df_cat_melt,             # Long-form data source for all facets.
    col='Variable',          # Create one column panel per variable name.
    col_wrap=3,              # Wrap after 3 panels per row.
    sharex=False,           # Let each panel have its own x-axis categorical labels.
    sharey=False            # Let each panel have its own y-axis scale.
)

g.map_dataframe(facet_countplot) # Apply the helper function to each facet.
g.set_xticklabels(rotation=45)   # Rotate x labels to reduce overlap.

plt.tight_layout() # Adjust spacing so labels/titles fit cleanly.
plt.show()         # Render the faceted plot.
```



Do you see anything of interest? Get into the habit of commenting on every table and figure you produce! In this case you might be looking to see if any variables have distributions you would not expect. One quick observation is that some of the distributions are much more uniform than would be expected in real data. For example, roughly half of the students not having internet access and an approximately even distribution of parent education levels seems unlikely in reality.

2.2.2 Quantitative Variables

Next we consider quantitative variables. We begin again by isolating quantitative variables in their own data frame `df_num`:

```
In [23]: # Isolate numerical variables as a new data frame
df_num = df.select_dtypes(include=["int64", "float64"]).copy()
df_num.head()
```

Out [23]:

	age	study_hours_per_week	attendance_rate	previous_score	final_score
0	15	25	63.8	41	67
1	15	2	54.7	83	28
2	19	10	90.5	73	49
3	16	26	66.8	75	70
4	15	25	73.0	67	77

!! Problem

Q2 Summary Statistics

Recall the `describe` function which produces a table of summary statistics for numerical variables. An example of this output (with values rounded to 2DP) is shown below:

In [24]: `df_num.describe().round(2)`

Out [24]:

	age	study_hours_per_week	attendance_rate	previous_score	final_s
count	500.00	500.00	500.00	500.00	50
mean	16.98	15.31	76.38	62.99	5
std	1.43	8.57	13.82	18.94	1
min	15.00	2.00	50.20	30.00	2
25%	16.00	8.00	64.47	46.00	4
50%	17.00	15.00	76.50	64.00	5
75%	18.00	23.00	88.52	79.00	6
max	19.00	30.00	100.00	95.00	9

Write a function `quantitative_summary` that produces a modified version of this output with the following rows added to the bottom:

1. `median` - the 50% percentile.
2. `range` - difference between the maximum and minimum.
3. `iqr` - difference between the upper and lower quartile.
4. `std` - the standard deviation (look up how to compute this in numpy).

If you need to, look up the pandas functions for computing these statistics. You can use `concat` to combine the describe output data frame with one you create

with these additional statistics. Depending on your construction you may need to transpose one of the data frames.

Solution:

```
In [27]: def quantitative_summary(df):
df_num = df.select_dtypes(include=["int64", "float64"])
desc = df_num.describe()

new_rows = {
    'median': df_num.median(),
    'range': df_num.max() - df_num.min(),
    'iqr': df_num.quantile(0.75) - df_num.quantile(0.25),
    'std': df_num.std()
}
new_rows_df = pd.DataFrame(new_rows)
results = pd.concat([desc, new_rows_df])
return results.round(2)

summary_table = quantitative_summary(df)
print(summary_table)
```

	age	study_hours_per_week	attendance_rate	\		
count	500.00	500.00	500.00			
mean	16.98	15.31	76.38			
std	1.43	8.57	13.82			
min	15.00	2.00	50.20			
25%	16.00	8.00	64.47			
50%	17.00	15.00	76.50			
75%	18.00	23.00	88.52			
max	19.00	30.00	100.00			
age	NaN	NaN	NaN			
study_hours_per_week	NaN	NaN	NaN			
attendance_rate	NaN	NaN	NaN			
previous_score	NaN	NaN	NaN			
final_score	NaN	NaN	NaN			
	previous_score	final_score	median	range	iqr	
std						
count	500.00	500.00	NaN	NaN	NaN	NaN
NaN						
mean	62.99	55.98	NaN	NaN	NaN	NaN
NaN						
std	18.94	15.37	NaN	NaN	NaN	NaN
NaN						
min	30.00	20.00	NaN	NaN	NaN	NaN
NaN						
25%	46.00	45.00	NaN	NaN	NaN	NaN
NaN						
50%	64.00	56.00	NaN	NaN	NaN	NaN
NaN						
75%	79.00	68.00	NaN	NaN	NaN	NaN
NaN						
max	95.00	95.00	NaN	NaN	NaN	NaN
NaN						
age	NaN	NaN	17.0	4.0	2.00	
1.43						
study_hours_per_week	NaN	NaN	15.0	28.0	15.00	
8.57						
attendance_rate	NaN	NaN	76.5	49.8	24.05	
13.82						
previous_score	NaN	NaN	64.0	65.0	33.00	
18.94						
final_score	NaN	NaN	56.0	75.0	23.00	
15.37						

As discussed in lecture, the standard plots used to perform univariate analysis on numerical variables are box plots (which visualize the quartiles and outliers) and histograms (which visualize the distribution shapes).

!! Problem

Q3 Multiplots

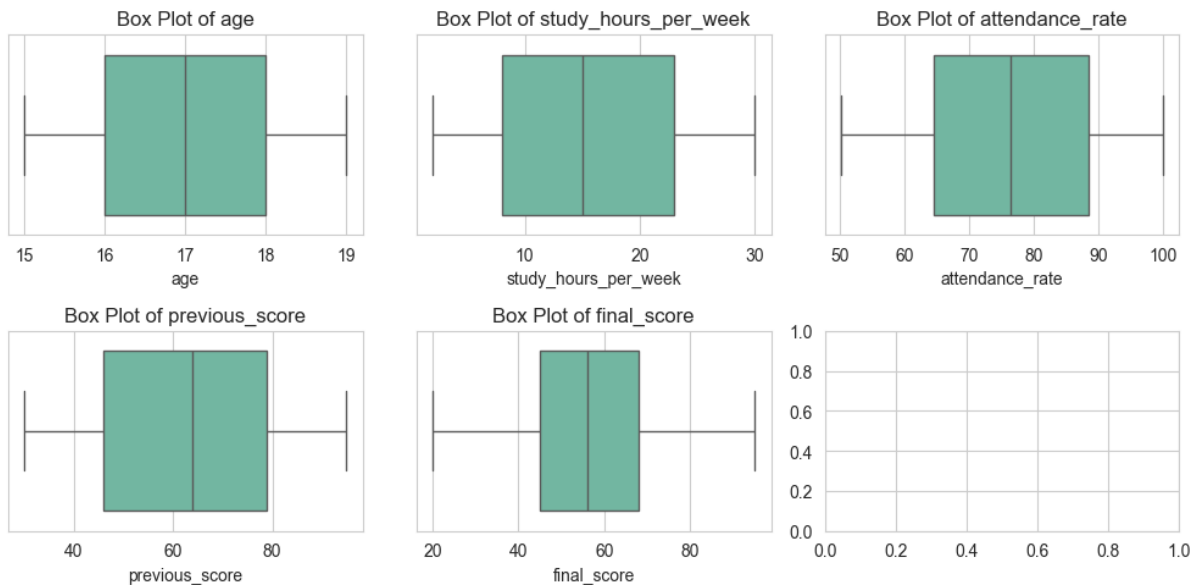
Using the count plot multiplot syntax as a guide produce two multiplots for the quantitative variables of our data set, one containing the box plots and the other histograms. Under each plot, briefly discuss your findings.

Solution:

```
In [28]: num_col = df.select_dtypes(include=["int64", "float64"]).columns
# Boxplot
fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(10, 5))
axes = axes.flatten()

for i, col in enumerate(num_col):
    sns.boxplot(data=df, x=col, ax=axes[i])
    axes[i].set(title=f'Box Plot of {col}', xlabel=col)

plt.tight_layout()
plt.show()
```



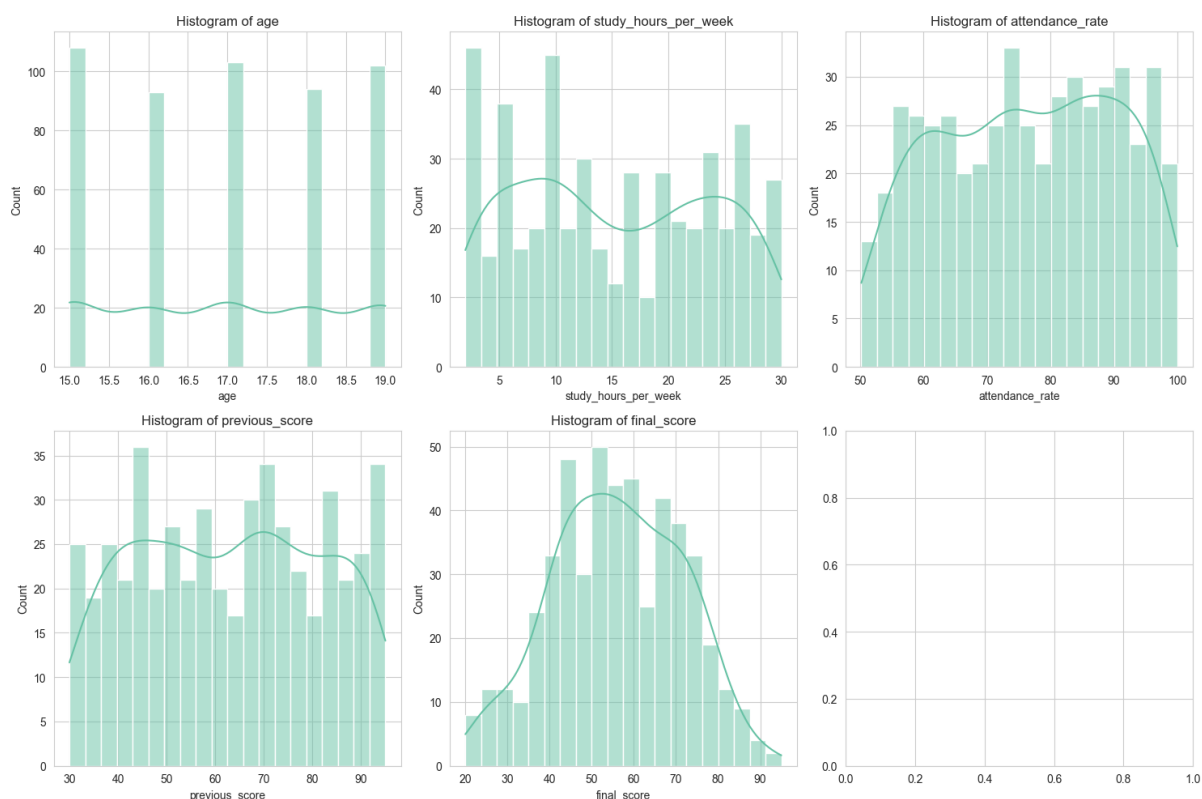
In []: This boxplot shows the spread of students performance with the IQR and be pretty symmetrical with no outliers.

```
In [29]: # Histogram
fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(15, 10))
axes = axes.flatten()

for i, col in enumerate(num_col):
    sns.histplot(data=df, x=col, bins=20, kde=True, ax=axes[i])
    axes[i].set(title=f'Histogram of {col}', xlabel=col, ylabel='Count')

plt.tight_layout()
```

```
plt.show()
```



In []: So this histogram makes my data look a lot less normal than I expected **except for** the Histogram of final score looks approximately normal.

Next week we will continue our exploration of this data set, focusing on multivariate analysis and specifically identifying any evidence of relationships with the variable of interest `final_score`. This will lead us nicely towards thinking about variables more formally as dependent and independent variables and considering potential models such as linear regression.

Submission Checklist

- Ensure you have completed all problems in the notebook.
- Clear your document code outputs and rerun the notebook, checking for any new errors.
- At the top of your notebook click more options -> export -> pdf.
 - Alternatively, export to html and save as a pdf.
- Make sure your pdf has all code chunks clearly visible and executed.
- Submit before the posted deadline.